

## 前言

### 01.学习概述

#### 1.1 学习目标

#### 1.2 前置知识

### 02.核心概念

#### 2.1 是什么？

#### 2.2 解决什么问题？

#### 2.3 基本特性

### 03.原理机制

#### 3.1 核心概念

#### 3.2 EGL核心方法\_其一：eglMakeCurrent

#### 3.3 EGL核心方法\_其二：eglSwapBuffers

#### 3.4 EGL代码实际应用 - 需求分析及架构设计

### 04.底层原理

### 05.深度思考

#### 5.1 关键问题探究

#### 5.2 设计对比

### 06.实践验证

#### 6.1 行为验证代码

#### 6.2 性能测试

### 07.应用场景

#### 7.1 最佳实践

#### 7.2 使用禁忌

### 08.总结提炼

#### 8.1 核心收获

#### 8.2 知识图谱

#### 8.3 延伸思考

### 09.参考资料

#### 其他介绍

##### 01.关于我的博客

## 前言

学习要符合如下的标准化链条：了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

## 01.学习概述

### • 学习主题：

### • 知识类型：

☐ ☒ Android/

☐ ☒ 01.基础组件与机制

☐ ☒ 四大组件

☐ ☒ IPC机制

☐ ☒ 消息机制

☐ ☒ 事件分发机制

☐ ☒ View与渲染体系（含Window、复杂控件、动画）

☐ ☒ 存储与数据安全（SharedPreferences/DataStore/Room/Scoped Storage）

- ☐ ☒ 02. 架构与工程化
  - ☐ ☒ 架构模式 (MVC/MVP/MVVM/MVI)
  - ☐ ☒ 依赖注入 (Koin/Hilt/Dagger)
  - ☐ ☒ 路由与模块化 (ARouter、Navigation)
  - ☐ ☒ Gradle与构建优化
  - ☐ ☒ 插件化与动态化
  - ☐ ☒ 插桩与监控框架
- ☐ ☒ 03.性能优化与故障诊断
  - ☐ ☒ ANR分析与优化
  - ☐ ☒ 启动耗时优化
  - ☐ ☒ 内存泄漏监控
  - ☐ ☒ 监控与诊断工具
- ☐ ☒ 04.Jetpack与生态框架
  - ☐ ☒ Room
  - ☐ ☒ Paging
  - ☐ ☒ WorkManager
  - ☐ ☒ Compose
- ☐ ☒ 05.Framework与系统机制
  - ☐ ☒ ActivityManagerService (含ANR触发机制)
  - ☐ ☒ Binder机制
- ☐ ☒ 音视频开发/
  - ☐ ☒ 01.基础知识
  - ☒ ☒ 02.OpenGL渲染视频
  - ☐ ☒ 03.FFmpeg音视频解码
- ☐ ☒ Java/
  - ☐ ☒ 01.基础知识
  - ☐ ☒ 02.集合框架
  - ☐ ☒ 03.异常处理
  - ☐ ☒ 04.多线程与并发
  - ☐ ☒ 06.JVM
- ☐ ☒ Kotlin/
  - ☐ ☒ 01.基础语法
  - ☐ ☒ 02.高阶扩展
  - ☐ ☒ 03.协程和流
- ☐ ☒ Flutter/
  - ☐ ☒ 01.基础知识
    - ☐ ☒ Dart 语言基础
    - ☐ ☒ Widget 基础与生命周期
    - ☐ ☒ Flutter 基础组件

- ☐ ☒ 布局与约束
- ☐ ☒ 绘制与渲染体系
- ☐ ☒ 状态管理
- ☐ ☒ 事件处理与手势系统
- ☐ ☒ 原生通信
- ☐ ☒ 02.路由与导航
- ☐ ☒ 03.性能优化与故障诊断
- ☐ ☒ 04.异步编程
- ☐ ☒ 05.项目经验与案例沉淀
- ☐ ☒ 自我管理/
  - ☐ ☒ 01.内观
- ☐ ☒ 项目经验/
  - ☐ ☒ 01.启动逻辑
  - ☐ ☒ 02.云值守
  - ☐ ☒ 03.智控平台
  - ☐ ☒ 04.视频巡店

- 学习来源：
- 重要程度：★★★★★
- 学习日期：2025.
- 记录人：@panruiqi

## 1.1 学习目标

- 了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

## 1.2 前置知识



## 02.核心概念

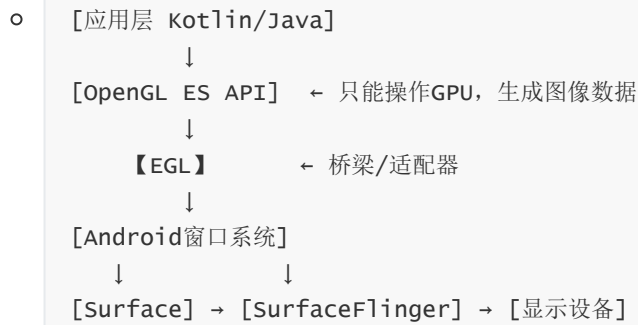
---

### 2.1 是什么？

想理解EGL，就得先理解OpenGL能做什么？

OpenGL本质是一组GPU操作API，他负责控制GPU执行图形渲染计算。这里存在一个问题：OpenGL控制GPU执行渲染操作了。渲染的结果存放在哪？如果我们想显示在屏幕上该如何显示呢？

- 所以EGL的作用就呼之欲出了，他是OpenGL渲染链路中的桥梁，用于连接OpenGL和Android窗口，让OpenGL渲染结果存放到什么位置，如果要显示交给谁去显示。



补充：

理解更新，后续我们的理解为：EGL不是我们上面理解的单纯的告知OpenGL，渲染的存放位置和显示的目标而已。他的职责是双重的，既有替OpenGL桥接窗口系统。又有替OpenGL管理Context

意义更新：同时这里也可以解释：为什么Android窗口和OpenGL的连接需要一个EGL做中间的适配层。因为OpenGL是跨平台的，但窗口系统是平台特定的，不同平台有自己独特的窗口系统，OpenGL根本不认识Android的窗口，所以需要EGL去封装底层的实现，向OpenGL提供平台无关的支持

## 2.2 解决什么问题？

解决OpenGL不知道渲染结果存放到什么位置，以及显示交给谁去显示的问题

## 2.3 基本特性

# 03.原理机制

## 3.1 核心概念

EGLDisplay - 显示设备的抽象

- 定义：告诉OpenGL渲染出的数据最终显示到哪？我们通过EGLDisplay可以连接到SurfaceFlinger（系统合成器）
- 本质：建立App进程和SurfaceFlinger进程的IPC连接
- 代码：`val display = EGL14.eglGetDisplay(EGL14.EGL_DEFAULT_DISPLAY)`

EGLConfig - 渲染格式配置

- 定义：告诉系统你需要什么样的渲染格式
- 本质：决定最终屏幕输出的像素格式
- 流程及代码示例：



```

    val attrList = intArrayOf(
        EGL14.EGL_RED_SIZE, 8,      // 红色通道8位
        EGL14.EGL_GREEN_SIZE, 8,    // 绿色通道8位
        EGL14.EGL_BLUE_SIZE, 8,     // 蓝色通道8位
        EGL14.EGL_ALPHA_SIZE, 8,    // 透明通道8位
        EGL14.EGL_RENDERABLE_TYPE, EGL14.EGL_OPENGL_ES2_BIT,
        EGL14.EGL_NONE
    )

    val configs = arrayOfNulls<EGLConfig>(1)
    EGL14.eglChooseConfig(display, attrList, 0, configs, 0, 1, numConfigs,
    0)
    // 系统返回最匹配的配置

```

## EGLSurface - 渲染目标缓冲区

- 定义：告诉系统，渲染出的数据存放到哪？（补充：后续理解，他的核心作用是封装原生的Surface，向上提供统一的，平台无关的eglSurface）
- 图形系统架构：先来理解一下Android的图形系统架构。Surface的本质是GraphicBuffer

```

    [Kotlin层] SurfaceView
      ↓
    [Native层] Surface → [指针] ANativeWindow
      ↓
    [系统层] BufferQueue (由SurfaceFlinger分配)
      ↓
    [内存] GraphicBuffer #0, #1, #2 (三缓冲)

```

- EGL在这里的位置：Surface是对底层的BufferQueue的封装，EGLSurface是对Surface的封装，通过EGLSurface，我们把默认FBO的输出设置为了这个Surface。

```

    // 1. Kotlin层创建SurfaceView
    val surfaceView = SurfaceView(context)

    // 2. 获取Native层的Surface
    val surface = surfaceView.holder.surface

    // 3. EGL绑定这个Surface，创建EGLSurface
    val eglSurface = EGL14.eglCreateWindowSurface(
        display, config,
        surface, // ← 将BufferQueue绑定到EGLSurface
        surfaceAttribs, 0
    )

```

- 可是，为什么不能直接设置呢，直接设置GPU的渲染的输出结果为Surface呢？为什么需要EGLSurface这个"中间层"？
  - 这里有一个核心矛盾：OpenGL ES是跨平台的，但窗口系统是平台特定的。不同平台有自己独特的窗口系统

- OpenGL ES (跨平台标准)

↓

??? ← 如何连接?

↓

不同平台的窗口系统:

- Android: Surface (BufferQueue)
- Windows: HWND (GDI/DirectX)
- Linux: Xwindow (X11)
- iOS: CAEAGLLayer

- 如果没有EGL, 我们直接设置窗口, 这会发生什么? OpenGL根本不认识Surface啊。

- // 假设OpenGL直接支持Android Surface (这是不可能的)  
GLES20.glSetRenderTarget(androidSurface) // ✗ OpenGL不认识Surface
- // 那在Windows上怎么办?  
GLES20.glSetRenderTarget(windowsHWND) // ✗ OpenGL也不认识HWND
- // Linux呢?  
GLES20.glSetRenderTarget(xwindow) // ✗ OpenGL还是不认识

- 好的, 这是EGL的解决方案, 封装原生的Surface, 向上提供统一的, 平台无关的eglSurface

- // 统一的跨平台API  
val eglSurface = EGL14.eglCreateWindowSurface(  
display, config,  
nativewindow, // ← 这个可以是任何平台的原生窗口  
attribs, 0  
)  
  
// 在Android上: nativewindow = Surface  
// 在Windows上: nativewindow = HWND  
// 在Linux上: nativewindow = Xwindow  
// OpenGL完全不需要知道底层是什么!

## EGLContext

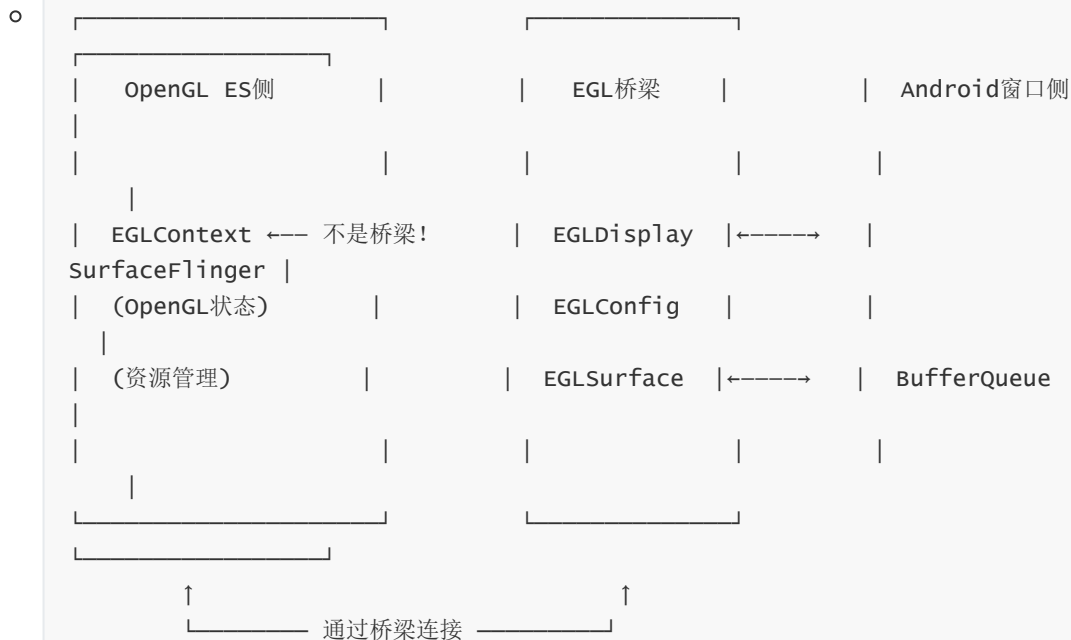
- 思考: 在我的思考中, EGLContext的作用似乎是多余的, 从EGL作为桥梁的角度看:

- 连接OpenGL和窗口系统只需要:
  - ☒ EGLDisplay → 连接到显示系统(SurfaceFlinger)
  - ☒ EGLConfig → 配置渲染格式(RGBA位数)
  - ☒ EGLSurface → 指定渲染目标(BufferQueue)
  - ? EGLContext → ??? 和"桥梁"有什么关系?

- 唯一的答案: EGLContext不是为了"桥接"而存在的! 那么EGLContext为了什么而存在呢?

- EGLContext存在的意义

- 看一份OpenGL, EGL, Android窗口的架构



- EGLContext是OpenGL自身的需求，不是桥梁的需求，EGL之所以管理Context，是因为没人替OpenGL做这件事
- 如果没有EGLContext会怎样
  - OpenGL状态冲突：OpenGL的状态被多个线程共享，渲染结果会互相干扰

```
// 假设没有Context，所有状态都是全局的

// 线程A：渲染视频1
GL20.glUseProgram(shaderProgram1)    // 设置着色器1
GL20.glBindTexture(textureId1)        // 绑定纹理1

// 此时，线程B突然执行
GL20.glUseProgram(shaderProgram2)    // 设置着色器2 ✗ 覆盖了线程A的状态！
GL20.glBindTexture(textureId2)        // 绑定纹理2 ✗ 覆盖了线程A的纹理！

// 线程A继续执行
GL20.glDrawArrays(...)                // ✗ 用的是shader2和texture2，画面错乱！
```

- OpenGL资源ID冲突

```
// 线程A创建纹理
val textureId = IntArray(1)
GL20.glGenTextures(1, textureId, 0)
// 返回 textureId = 1

// 线程B也创建纹理
val textureId2 = IntArray(1)
GL20.glGenTextures(1, textureId2, 0)
// 也返回 textureId2 = 1 ✗ 冲突了！

// 问题：如果没有Context隔离，纹理ID会冲突
```

- EGLContext定义

- 好，我们上面讨论了EGLContext解决了什么问题，探讨了他的必要性，现在来看看EGLContext是什么？
- 我们先来看看EGLContext如何解决OpenGL的并发使用吧

- OpenGL就像一个"绘画工具箱":
  - 有各种画笔(Shader)
  - 有各种颜料(Texture)
  - 有画布(FBO)
  - 有当前选中的工具(状态：当前激活的纹理单元、混合模式等)

如果多个人(线程)共用一个工具箱：

- ✗ A正在用红色画笔，B突然换成蓝色，A画出来就是蓝色
- ✗ A的画布被B弄乱了
- ✗ 大家的工具互相干扰

解决方案：每个人有自己的工具箱！

- ☑ Context A = 画家A的工具箱
- ☑ Context B = 画家B的工具箱

- 好，如果我们把OpenGL比作一个CPU资源，那么这个并发使用像不像多线程，这个EGLContext像不像线程上下文？那他应该包含什么呢？
- 当前OpenGL执行的状态，所用到的资源，绑定的线程，渲染的目标Surface等。

- ```
// EGLContext内部结构（简化）
class EGLContext {
    // 1. OpenGL状态
    struct State {
        GLuint currentProgram;           // 当前着色器程序
        GLuint currentTexture[32];       // 每个纹理单元的纹理
        GLuint currentFramebuffer;       // 当前FBO
        GLuint currentVertexArray;       // 当前VAO
        GLenum blendMode;                // 混合模式
        GLfloat clearColor[4];           // 清屏颜色
        // ... 还有上百个状态
    } state;

    // 2. 资源池（这些资源只在这个Context内有效）
    struct Resources {
        Map<GLuint, Texture> textures;    // 纹理对象
        Map<GLuint, Shader> shaders;     // 着色器对象
        Map<GLuint, Buffer> vbos;        // 顶点缓冲对象
        Map<GLuint, Framebuffer> fbos;   // 帧缓冲对象
        // ...
    } resources;

    // 3. 绑定信息
    Thread* boundThread;                // 绑定到哪个线程
    EGLSurface* drawSurface;             // 渲染目标Surface
    EGLSurface* readSurface;             // 读取源Surface
};
```

- 再回过头来看一下，OpenGL为何自身没有提供管理机制呢？
  - 这是因为，OpenGL最初设计时，Context的创建和管理是平台相关的。而OpenGL又是通用的，所以需要EGL进行适配，封装原生的上下文信息，向上提供统一的，平台无关的EGLContext



- ok, 从这我们可以看出, EGL不是我们上面理解的单纯的告知OpenGL, 渲染的存放位置和显示的目标而已。他的职责是双重的, 既有替OpenGL桥接窗口系统。又有替OpenGL管理Context

- EGL的两大职责:
  1. 桥接OpenGL和窗口系统 (你理解的部分)
    - EGLDisplay
    - EGLConfig
    - EGLSurface
  2. 替OpenGL管理Context (容易被忽略的部分)
    - EGLContext

## 3.2 EGL核心方法\_其一: eglMakeCurrent

引子: 我们先来看一个问题

- 如果同时有两个GLSurfaceView在渲染视频画面, OpenGL为什么能够正确的把画面分别绘制到两个GLSurfaceView中?

- ```
// 场景: 两个SurfaceView同时渲染

// GLSurfaceView A - 渲染视频1
glSurfaceViewA.queueEvent {
    GLES20.glClearColor(1.0f, 0.0f, 0.0f, 1.0f) // 红色
    GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, textureA)
    GLES20.glDrawArrays(...)
}

// GLSurfaceView B - 渲染视频2
glSurfaceViewB.queueEvent {
    GLES20.glClearColor(0.0f, 1.0f, 0.0f, 1.0f) // 绿色
    GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, textureB)
    GLES20.glDrawArrays(...)
}
```

- OpenGL的API没有指定"渲染到哪个SurfaceView"; GLES20.glDrawArrays()没有参数说明目标是A还是B; 为什么画面不会画错?

eglMakeCurrent方法

- 看看方法签名

- ```
fun eglMakeCurrent(
    display: EGLDisplay,      // EGL显示连接
    draw: EGLSurface,        // 绘制目标Surface
    read: EGLSurface,        // 读取源Surface (通常和draw相同)
    context: EGLContext      // OpenGL上下文
): Boolean
```

- 作用: 实现了设备显示窗口 (EGLDisplay)、OpenGL上下文 (EGLContext)、图像数据缓存 (EGLSurface)、当前线程的绑定, 将这些数据放入到线程的TLS中。
- 完整流程: 这里最关键的是步骤2, 我感觉可以简单理解为把EGLContext存放到TLS中。

- ```
调用 eglMakeCurrent(display, surfaceA, surfaceA, contextA)
      ↓
    【步骤1】获取当前线程ID
```

↓

【步骤2】将以下信息存储到线程局部存储(TLS):

- EGLDisplay: display
- EGLContext: contextA
- draw Surface: surfaceA
- read Surface: surfaceA

↓

【步骤3】将contextA标记为"绑定到当前线程"

↓

【步骤4】将contextA的默认FBO(ID=0)关联到surfaceA

↓

【结果】当前线程与OpenGL渲染环境建立了绑定关系

TLS机制:

- 什么是TLS: TLS: Thread Local Storage (线程局部存储)。线程的私有存储空间
- OpenGL如何利用TLS?
  - 在上面绑定之后, 只要你是在渲染线程中调用任何OpenGL ES的API (比如生产纹理ID的方法GLes20.glGenTextures), OpenGL会自动根据当前线程, 切换上下文 (也就是切换OpenGL的渲染信息和资源)。
- 实现原理
  - 每个OpenGL API调用时, 都会先从TLS获取当前线程的Context
  - 所有的状态、资源、渲染目标都从这个Context中获取
  - // OpenGL ES API的内部实现 (伪代码)

```
void glDrawArrays(GLenum mode, GLint first, GLsizei count) {
    // 【关键】第1步: 获取当前线程的EGLContext
    EGLContext* context = eglGetCurrentContext(); // 从TLS读取

    if (context == NULL) {
        // 没有Context, 报错
        error("No EGLContext bound to current thread!");
        return;
    }

    // 第2步: 从Context获取渲染目标
    EGLSurface* surface = context->drawSurface;

    // 第3步: 从Context获取当前状态
    GLuint program = context->state.currentProgram;
    GLuint texture = context->state.currentTexture[0];

    // 第4步: 从Context获取资源
    Shader* shader = context->resources.shaders[program];
    Texture* tex = context->resources.textures[texture];

    // 第5步: 执行实际的渲染
    GPU.render(surface, shader, tex, ...);
}
```

现在来回答两个问题:

1. 多窗口渲染时, 两个GLSurfaceView为什么不会画错

- 线程A调用 GLes20.glDrawArrays()

```

↓
OpenGL内部: getCurrentThreadId() → Thread A
↓
从TLS读取: Thread A → { contextA, surfaceA }
↓
使用contextA的资源 → 渲染到surfaceA
↓
显示在GLSurfaceView A ☒

线程B调用 GLES20.glDrawArrays()
↓
OpenGL内部: getCurrentThreadId() → Thread B
↓
从TLS读取: Thread B → { contextB, surfaceB }
↓
使用contextB的资源 → 渲染到surfaceB
↓
显示在GLSurfaceView B ☒

```

## 2. 为什么必须在OpenGL线程中调用API?

- 如果你在非调用glMakeCurrent的线程中去调用OpenGL的API，系统将找不到对应的OpenGL上下文，也就找不到对应的资源，可能会导致异常出错

```

◦ // 主线程
val mainThread = Thread.currentThread()

// OpenGL线程
val glThread = Thread {
    // 初始化EGL
    val context = EGL14.eglCreateContext(...)
    val surface = EGL14.eglCreateWindowSurface(...)

    // 绑定到glThread
    EGL14.eglMakeCurrent(display, surface, surface, context)
    // → glThread的TLS: { context, surface }

    // 创建纹理
    val textureId = IntArray(1)
    GLES20.glGenTextures(1, textureId, 0) // ☒ 成功, textureId = 1
}
glThread.start()

// 在主线程中尝试使用这个纹理
Thread.sleep(1000) // 等待GL线程初始化完成
GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, 1) // ✗ 错误!

// 为什么错误?
// 1. OpenGL查找主线程的TLS → NULL (主线程没有调用eglMakeCurrent)
// 2. 找不到Context, 找不到纹理资源
// 3. 可能崩溃或者不生效

```

- 正确的做法

- ```

// 方式1: 在OpenGL线程中执行
glSurfaceView.queueEvent {
    // 这个代码块在GLThread中执行
    GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, textureId) // ☑
}

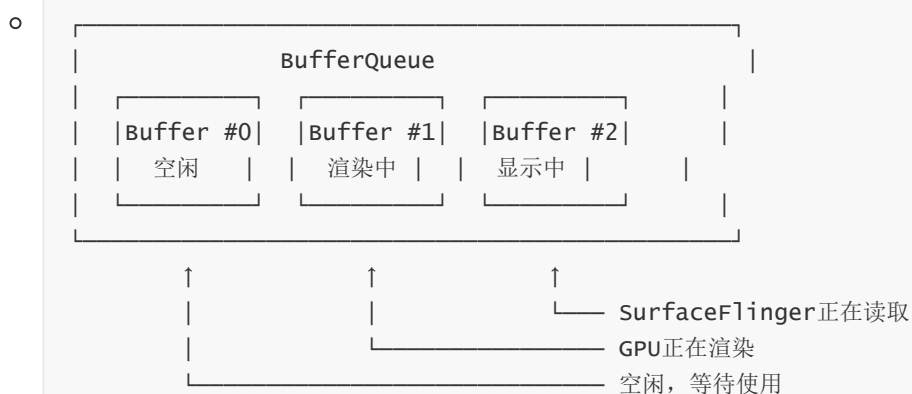
// 方式2: 共享Context (高级用法)
val sharedContext = EGL14.eglCreateContext(
    display, config,
    mainContext, // 共享主Context的资源池
    attribs, 0
)
// 这样两个Context可以访问同一批纹理

```

### 3.3 EGL核心方法\_其二: eglSwapBuffers

eglSwapBuffers - 显示到屏幕

- 作用: 将EGLSurface数据显示到设备屏幕上的方法。在OpenGL绘制完图像化, 只有调用该方法, 才能真正显示出来。
- 大致原理: 交互前后缓存, Binder机制通知SurfaceFlinger消费当前渲染完成的帧到屏幕上。
  - 先看当前的BufferQueue的状态 (Buffer1是后缓冲存放GPU渲染的结果; Buffer2是前缓冲显示在屏幕上)



- eglSwapBuffers的执行步骤, 本质是让SurfaceFlinger获取渲染中的FrameBuffer, 将其渲染出来。

- ```

// EGL14.eglSwapBuffers(display, surface) 内部流程

bool eglSwapBuffers(EGLDisplay display, EGLSurface surface) {

    // ===== 第1步: 等待GPU渲染完成 =====
    glFinish(); // 确保所有OpenGL命令执行完毕
    // 此时Buffer #1的渲染已完成

    // ===== 第2步: 解锁当前Buffer =====
    GraphicBuffer* currentBuffer = surface->currentBuffer; // Buffer
#1
    currentBuffer->unlock();
    // GPU不再写入这个Buffer

    // ===== 第3步: 将Buffer标记为"ready" =====
    surface->bufferQueue->queueBuffer(currentBuffer);
    // Buffer #1状态: 渲染中 → 已完成(ready)

```

```

// ===== 第4步: 通知SurfaceFlinger =====
// 通过Binder IPC发送信号
notifySurfaceFlinger(surface->layer, currentBuffer);
// 告诉SurfaceFlinger: "我有新的一帧ready了"

// ===== 第5步: 等待VSync信号 (可选) =====
if (surface->swapInterval > 0) {
    waitForVSync(); // 等待垂直同步信号
}

// ===== 第6步: 获取下一个空闲Buffer =====
GraphicBuffer* nextBuffer = surface->bufferQueue->dequeueBuffer();
// 从队列获取Buffer #0 (空闲)

// 可能会阻塞! 如果没有空闲Buffer, 等待
// 比如: SurfaceFlinger还没消费完Buffer #2

// ===== 第7步: 锁定新Buffer =====
void* address = nextBuffer->lock(WRITE);
// GPU可以开始往这个地址写入

// ===== 第8步: 更新EGLSurface状态 =====
surface->currentBuffer = nextBuffer; // 切换到Buffer #0
GPU.setRenderTarget(address);        // 告诉GPU新的渲染地址

return true;
}

```

## 双缓冲机制

- 如果没有双缓冲机制, 假设我们直接渲染到屏幕:

- 场景: 渲染一个移动的小球的一帧

时间线:

T1: 显示器扫描第1行 ----> 读取屏幕内存, 显示小球在位置A

T2: GPU开始渲染新一帧 ----> 更新内存, 小球移动到位置B

T3: 显示器扫描第100行 ----> 读取屏幕内存, 显示小球在位置B

T4: 显示器扫描第200行 ----> 继续显示小球在位置B

结果: 当前一帧占满整个屏幕, 但是画面撕裂

屏幕上半部分显示位置A的小球

屏幕下半部分显示位置B的小球

→ 画面被"撕裂"了!

- 一帧可能如下



- 双缓冲的解决方案, 一块渲染, 一块缓冲

- 使用两块内存：

前缓冲(Front Buffer)：显示器正在读取，显示在屏幕上  
后缓冲(Back Buffer)： GPU正在渲染，用户看不到

工作流程：

1. GPU在后缓冲中完整绘制一帧
2. 等待显示器扫描完成(VSync)
3. 交换前后缓冲的指针
4. 新画面显示，继续在新的后缓冲绘制

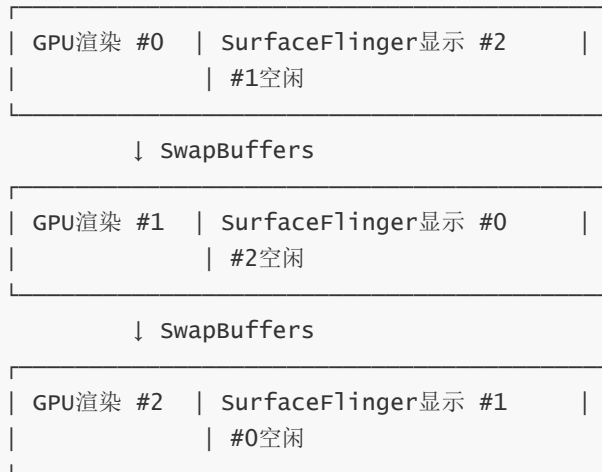
关键：显示器始终读取完整的一帧，不会看到"半成品"

### 看看三缓冲

- 双缓冲的问题：可能出现，Buffer #0: SurfaceFlinger正在显示（不能用）； Buffer #1: SurfaceFlinger刚显示完，可能还没释放。也就是：前缓冲还没来得及释放，但是后缓冲已经被使用了。
- 解决方案：来一个备用的

- 有三个Buffer： #0、#1、#2

循环流程：



关键：始终有一个空闲Buffer，GPU不会阻塞

### 看看VSync垂直同步机制

- 什么是VSync：就是显示器扫完一帧，准备到下一帧时发出的垂直同步中断信号。

- 显示器的工作方式：
  - 60Hz显示器：每秒刷新60次
  - 每次刷新间隔：16.67ms
  - 扫描方式：从上到下，从左到右

VSync信号：

- 显示器扫描完一帧，准备扫描下一帧时发出的信号
- 告诉系统："我准备好显示新画面了"

- 对应上面的EGL，等待VSync后获得后缓冲，让GPU写入，写入完成后可以 进行swap

-

```
// ===== 第5步: 等待VSync信号 (可选) =====
if (surface->swapInterval > 0) {
    waitForVSync(); // 等待垂直同步信号
}

// ===== 第6步: 获取下一个空闲Buffer =====
GraphicBuffer* nextBuffer = surface->bufferQueue->dequeueBuffer();
// 从队列获取Buffer #0 (空闲)

// 可能会阻塞! 如果没有空闲Buffer, 等待
// 比如: SurfaceFlinger还没消费完Buffer #2

// ===== 第7步: 锁定新Buffer =====
void* address = nextBuffer->lock(WRITE);
// GPU可以开始往这个地址写入

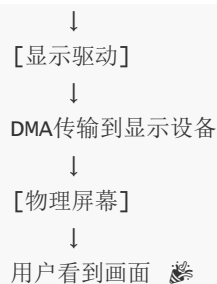
// ===== 第8步: 更新EGLSurface状态 =====
surface->currentBuffer = nextBuffer; // 切换到Buffer #0
GPU.setRenderTarget(address);       // 告诉GPU新的渲染地址

return true;
```

ok, 我们来总结一下帧渲染的原理和流程

- 如下





### 3.4 EGL代码实际应用 - 需求分析及架构设计

目的：不使用GLSurfaceView，手动管理EGL，实现OpenGL渲染到SurfaceView

- 核心需求：
  - 手动初始化EGL环境
  - 独立渲染线程，不阻塞主线程
  - 处理Surface生命周期（创建、变化、销毁）
  - 循环渲染机制（类似onDrawFrame）
  - 资源管理和清理

整体架构设计

- 四层架构全景图



- 没看懂，来吧，逐层讲解

第1层：EGLPlayerActivity

-



```

○ class EGLPlayerActivity : AppCompatActivity() {

    private val mRenderer = CustomerGLRenderer() // 创建渲染器

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_egl_player)

        initFirstVideo() // 初始化第一个视频
        initSecondVideo() // 初始化第二个视频（画中画）
        setRenderSurface() // 绑定SurfaceView
    }

    private fun initFirstVideo() {
        // 步骤1: 创建绘制器
        val drawer = VideoDrawer()
        drawer.setVideoSize(1920, 1080)

        // 步骤2: 获取SurfaceTexture, 用于视频解码
        drawer.getSurfaceTexture { surfaceTexture ->
            initPlayer(path, Surface(surfaceTexture), true)
        }

        // 步骤3: 添加到Renderer
        mRenderer.addDrawer(drawer)
    }

    private fun setRenderSurface() {
        // 关键: 将SurfaceView绑定到Renderer
        mRenderer.setSurface(sfv) // sfv是布局中的SurfaceView
    }
}

```

第2层: CustomerGLRenderer (协调层)

## 04.底层原理

---

## 05.深度思考

---

### 5.1 关键问题探究

### 5.2 设计对比

## 06.实践验证

---

### 6.1 行为验证代码

## 6.2 性能测试

## 07.应用场景

---

### 7.1 最佳实践

### 7.2 使用禁忌

## 08.总结提炼

---

### 8.1 核心收获

### 8.2 知识图谱

### 8.3 延伸思考

## 09.参考资料

---

- 1.
- 2.
- 3.

## 其他介绍

---

### 01.关于我的博客

- csdn: [http://my.csdn.net/qg\\_35829566](http://my.csdn.net/qg_35829566)
- 掘金: <https://juejin.im/user/499639464759898>
- github: <https://github.com/jjjjjjava>
- 邮箱: [[934137388@qq.com](mailto:934137388@qq.com)]